

Autoreject and RANSAC in the MEG/EEG Preprocessing Pipeline

1. The Overall Goal

The *autoreject* algorithm (Jas et al., 2017) automates the identification and handling of artifacts in MEG and EEG recordings.

It detects **bad channels** (persistently noisy sensors) and **bad epochs** (trials or segments corrupted by transient artifacts) and then either repairs them through interpolation or rejects them.

Your preprocessing pipeline uses two complementary tools:

- **RANSAC** – finds *globally bad sensors* that are unreliable across the entire recording.
- **Autoreject (local)** – finds *trial-specific bad sensors* and *bad epochs*, learning thresholds directly from the data.

Together they create a reproducible, physics-based, and statistically principled data-cleaning step.

2. Identifying Bad Channels

2.1 RANSAC: Persistent Global Failures

RANSAC (*Random Sample Consensus*) repeatedly samples small subsets of sensors (typically 25 % of the array), reconstructs each sensor's signal from its neighbors via spatial interpolation, and compares the predicted versus real signal.

If the correlation between the prediction and the true signal is consistently low (≈ 0.75 threshold for $> 40\%$ of time), the channel is marked as **globally bad** and interpolated once.

RANSAC is ideal for continuous data; it catches broken leads, flat lines, and chronic noise.

2.2 Autoreject's Global Channel Criterion

Autoreject monitors how often each sensor exceeds its learned threshold across trials.

If a sensor is bad in more than a set fraction of epochs—controlled by `global_epoch_thresh`—it is flagged as **globally bad** and excluded from further interpolation or ICA.

Example setting (shown here as plain text, not fenced YAML):

```
global_epoch_thresh: 0.5    # mark as globally bad if >50% of epochs are bad
```

Thus:

- **RANSAC** detects global failures from continuous data.
- **Autoreject** detects global failures by tallying repeated trial-level failures.

Both can remove sensors from future interpolation and ICA.

3. Detecting and Handling Bad Epochs

After stable sensors are determined, *autoreject (local)* focuses on identifying **bad epochs**—trials in which one or more sensors show transient artifacts.

Each sensor receives its own peak-to-peak amplitude threshold, learned automatically through cross-validation.

4. How Thresholds Are Learned (Cross-Validation and Folds)

Unlike fixed rules (e.g., “reject > 150 μV ”), *autoreject* learns amplitude thresholds τ_j from data.

1. Split the epochs into K folds (**cv_folds**, typically 5–10).
2. For each candidate threshold τ :
 - **Training**: drop epochs exceeding τ and compute the average.
 - **Validation**: compute the median of held-out epochs (robust to outliers).
 - **Error**: calculate RMSE between training average and validation median.
3. Choose τ that minimizes average RMSE across folds.

This procedure guarantees thresholds that generalize to unseen data rather than overfitting specific trials.

5. Optimization Strategies (**thresh_method**)

5.1 Random Search

- Explores a fixed grid or random samples of τ values.
- **Pros**: simple, deterministic, transparent.
- **Best for**: small EEG datasets or debugging.
- **Cons**: slower convergence on large, high-density arrays.

5.2 Bayesian Optimization

- Builds a probabilistic (Gaussian-process) model of how RMSE changes with τ , selecting the most promising values iteratively.
- **Pros**: faster and more precise for many sensors; ideal for MEG.
- **Best for**: large or multi-subject pipelines.
- **Cons**: slightly more complex; may vary with random seed (fix **random_state_global** for reproducibility).

Guideline:

Use *Bayesian optimization* when efficiency matters (typical for MEG).

Use *Random search* for deterministic runs or smaller EEG data.

6. Repair vs Rejection – ρ and κ Parameters

After thresholds are learned, *autoreject* decides per epoch whether to repair or reject.

- **ρ (rho)** – maximum number of sensors that can be repaired (interpolated).
- **κ (kappa)** – fraction of sensors allowed bad before rejecting the epoch.

Typical configuration (plain text example):

```
n_interpolate: [0, 1, 2, 3]    # candidate  $\rho$  values; 0 disables repair
consensus_thresh: 0.3         #  $\kappa$  fraction; reject if >30% sensors are bad
```

If the number of bad sensors $> \kappa \times N$, the epoch is a **global bad epoch** and dropped.

Otherwise, up to ρ of the worst sensors are spatially interpolated.

To disable repairs entirely (reject only):

```
n_interpolate: [0]
```

7. YAML Parameter Overview

Full block for reference (copy directly into your YAML):

```
autoreject:
  enabled: true
  which_types: [mag, grad]
  filter:
    highpass: 1.0
    lowpass: 40.0
    resample_hz: 200
  epoch:
    duration: 2.0
    tmin: 0.0
    tmax: 2.0
  cv_folds: 7
  thresh_method: "bayesian_optimization"    # or "random_search"
  n_interpolate: [0, 1, 2, 3]                #  $\rho$  candidates; [0] disables
  repair
    consensus_thresh: 0.3                    #  $\kappa$  fraction for rejecting
  trials
    global_epoch_thresh: 0.5                 # mark channel bad if >50%
  epochs fail
    verbose: true
```

8. Practical Workflow

1. **Run RANSAC** (optional) to remove globally bad sensors.

2. **Epoch the data** around event triggers.
3. **Run Autoreject:**
 - Learns thresholds per sensor via cross-validation.
 - Repairs $\leq \rho$ bad sensors per epoch.
 - Rejects epochs $> \kappa$ bad sensors.
 - Marks channels failing $> \text{global_epoch_thresh}$ as globally bad.
4. **Proceed with ICA/SSP or averaging** on the cleaned dataset.

9. Practical Guidance

- For large MEG datasets → use Bayesian optimization (fast and robust).
- For small EEG datasets → use Random search (fixed and reproducible).
- To be conservative → lower `consensus_thresh` (reject more).
- To be lenient → increase `consensus_thresh` and allow higher ρ .
- Disable repair completely with `n_interpolate: [0]`.
- Always inspect diagnostic plots: black = clean, blue = repaired, red = rejected (Fig. 8 in Jas et al., 2017).

10. Conceptual Summary

Concept	Description
RANSAC	Detects globally bad sensors via spatial-consistency checks.
Autoreject	Detects and repairs locally bad sensors and epochs using data-driven thresholds.
Cross-validation	Learns thresholds that generalize across trials.
ρ (<code>n_interpolate</code>)	Max sensors per epoch that may be repaired.
κ (<code>consensus_thresh</code>)	Fraction of bad sensors allowed before epoch rejection.
<code>global_epoch_thresh</code>	Flags sensors that fail too often.
<code>thresh_method</code>	Chooses between Bayesian (efficient) and Random (simple) search.

Autoreject and RANSAC together form a transparent, reproducible, and interpretable artifact-rejection framework.

All decisions—repair vs rejection, efficiency vs determinism, strictness vs leniency—are fully controlled through parameters explicitly defined in your YAML configuration.